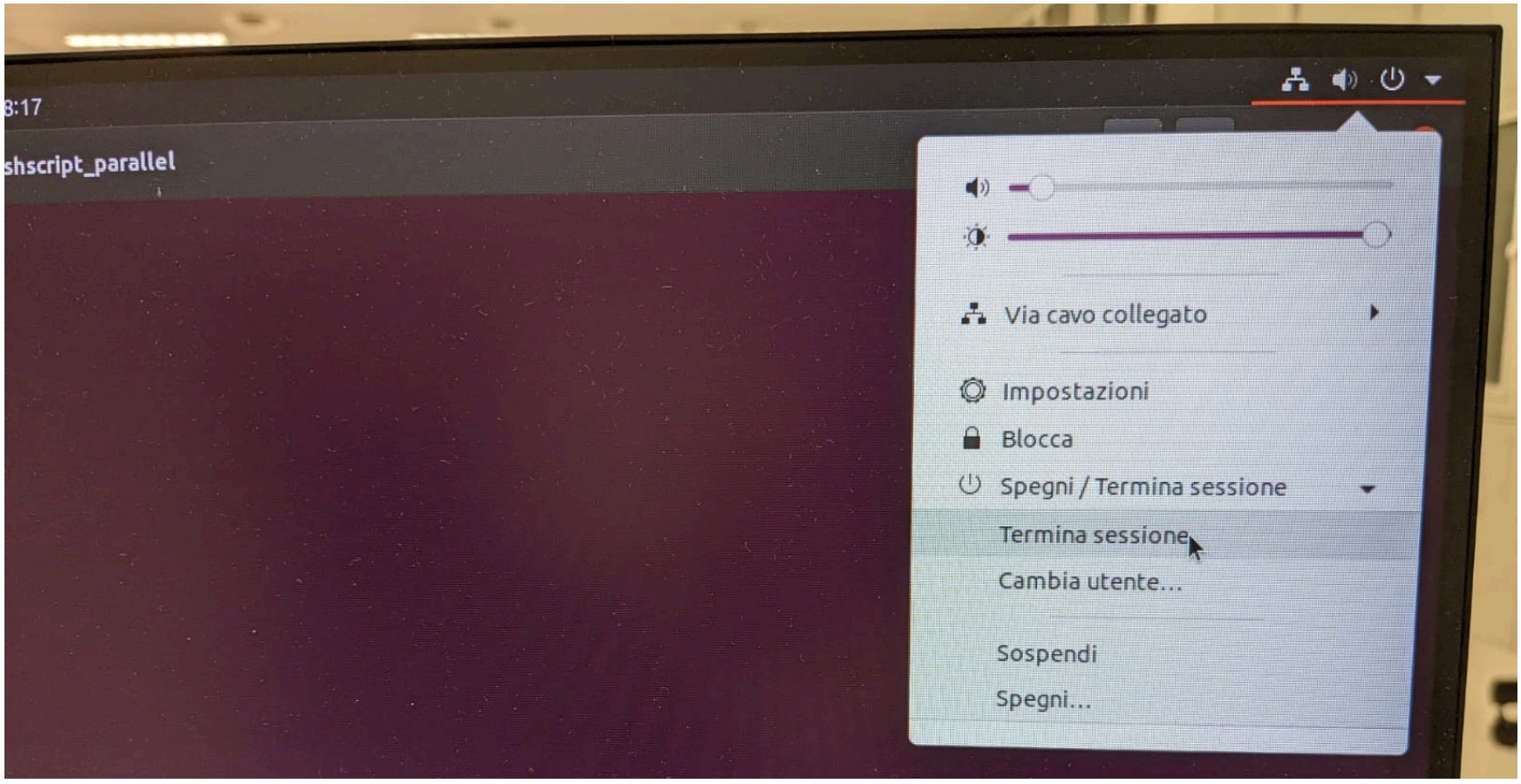
Corso di Sistemi Operativi

Corso estinto di Sistemi Operativi e Reti - Modulo Sistemi Operativi

Prova scritta - Settembre 2026

LEGGI ATTENTAMENTE LE ISTRUZIONI:

1. **Rinomina** subito la cartella chiamata "Cognome-Nome-Matricola" che hai trovato sul Desktop e in cui hai trovato questa traccia, sostituendo "Cognome" "Nome" e "Matricola" con i tuoi dati personali e **lasciando i trattini senza spazi**; se hai un doppio nome oppure un doppio cognome dovrai chiamare la cartella come in questo esempio:
    a. DeLuca-MarcoGiovanni-199999
2. **Carica** tutto il materiale didattico che vorrai usare sul Desktop; puoi farlo solo nei primi 5 minuti della prova;
3. **Svolgi** il compito; lascia tutto il sorgente che hai prodotto nella cartella di cui al punto 1;
4. Quando hai finito fai "**Termina Sessione/Logout**", ma lascia la postazione mantenendo il PC acceso.
5. **E' tua diretta responsabilità** garantire l'integrità del tuo elaborato, anche in caso di assenza di corrente. **Salva spesso il tuo lavoro**



e NON spegnere il PC.

SALVA SPESSO - SALVA SEMPRE

CI SONO DEI PUNTI AMBIGUI NELLA TRACCIA? DECIDI TU COSA FARE

È parte integrante di questo esercizio completare le specifiche date nei punti non esplicitamente definiti, introducendo nuove strutture dati, o estendendo quelle preesistenti laddove si ritenga necessario, risolvendo eventuali ambiguità. Si può cambiare il codice dei metodi esistenti dove serve.

POSSO CAMBIARE IL PROTOTIPO DEI METODI RICHIESTI O DI QUELLI ESISTENTI? Si, ma solo se li lasci retrocompatibili

Il codice che scriverai è pensato per essere usato come un modulo di libreria. Un modulo di libreria potrebbe essere usato da altri programmatori, i quali si aspettano di trovare una specifica interfaccia.

Se quindi modifichi il prototipo dei metodi pubblici di una classe, devi assicurarti che possano essere comunque utilizzati per come erano stati pensati prima delle tue modifiche. Puoi aggiungere qualsivoglia campo e metodo privato, e qualsivoglia classe ausiliaria. Analogamente, i metodi esistenti possono essere modificati nel loro codice, ma non se ne deve cambiare il risultato finale o il significato a meno che non ti venga richiesto esplicitamente.

CHE LINGUAGGIO POSSO USARE? PYTHON 3.X

POSSO CONSENTIRE SITUAZIONI DI RACE CONDITION NEL MIO CODICE? NO
POSSO CONSENTIRE SITUAZIONI DI DEADLOCK NEL MIO CODICE? NO
POSSO CONSENTIRE ALTRE SITUAZIONI DI BLOCCO TOTALE NEL MIO CODICE, TIPO NESTED LOCKOUT, LIVELOCK O ALTRO? NO
POSSO CONSENTIRE SITUAZIONI DI STARVATION NEL MIO CODICE? SI, tranne quando ti viene chiesto esplicitamente di rimuoverle
MA IL MAIN() LO DEVO AGGIORNARE? E I THREAD DI PROVA? SI

E' obbligatorio implementare esplicitamente il codice di prova oppure modificare il codice di prova pre-esistente, e accertarsi che giri senza errori prima della consegna.

ESERCIZIO 1 - PROGRAMMAZIONE MULTITHREADED

(Punteggio minimo richiesto 18/30. Pesa per ⅔ del voto finale)

Svolgi tutti i punti della traccia in un unico file che chiamerai `SOLUZIONE.py`
`-> Qualsiasi altro nome di file verrà IGNORATO <-`

Punto 1
Progetta un nuovo tipo di `Auto` che chiamerai `Auto_Ciclica` per come segue. L'auto ciclica deve fare N attraversamenti dell'incrocio prima di terminare, anziché uno solo; tra un attraversamento e l'altro devono passare T millisecondi; prima di ogni attraversamento la direzione di provenienza e l'intenzione di svolta devono cambiare, venendo scelte a caso. Fai in modo che nel main si possa condizionalmente avviare lo scenario standard oppure un nuovo scenario di test con sia auto cicliche che auto normali. Puoi semplicemente fare questo usando una variabile `num_scenario` in dipendenza della quale decidi cosa fare.

Punto 2
Progetta un nuovo scenario di simulazione dove ci sono due incroci A e B. Devi fare in modo che il lato Est di A e il lato Ovest di B siano collegati. In altre parole, quando un auto entra esce dal lato E di A, poco dopo deve entrare nel lato O di B. Viceversa, se un auto esce dal lato O di B, poco dopo deve entrare nel lato E di A. Per fare questo dovrai progettare un nuovo tipo di Auto che chiamerai `Auto_Doppio_Incrocio`, e aggiungere quant'altro ti serve per implementare questo punto. Aggiungi questo scenario, che prevede un doppio incrocio e solo Auto a Doppio Incrocio, a quelli possibili che si possono lanciare dal main.

Punto 3
Estendi il visualizzatore in maniera tale da poter stampare correttamente lo scenario di cui al punto 2.

Punto 4
Fai in modo che quando un incrocio viene liberato da un auto, vengano risvegliate solo le auto che potrebbero veramente beneficiare dei quadranti che si liberano.

SALVA SPESSO - SALVA SEMPRE

ESERCIZIO 2 - LINGUAGGI DI SCRIPTING

(Punteggio minimo richiesto 18/30. Pesa per ⅓ del voto finale)

Trovi allegato a questa traccia uno script Perl che presenta errori di sintassi e/o logici. Sistemalo per ottenere il comportamento che ti descriviamo sotto. ***NON devi stravolgere il contenuto dello script o riscriverlo da capo: la sua struttura deve rimanere pressoché invariata. E' prevista una penalità se correggi parti di codice che invece non presenta bug.***

Lo script `findMatch.pl` dovrebbe comportarsi come segue: dato un prefisso testuale, cerca ricorsivamente file e cartelle il cui nome inizia con quel prefisso, a partire da una cartella di riferimento, e ne stampa il path assoluto.

Lo script riceve in input i seguenti argomenti, tutti nominali (--nome=valore):

1. (OBBLIGATORIO) `--prefix=PREFIX`
   il prefisso da cercare tra i nomi di file e cartelle.

2. (OPZIONALE) `--path=PATH`
   la cartella da cui partire con la ricerca. Se l'utente non lo specifica, di default la ricerca parte dalla cartella corrente.

3. (OPZIONALE) `--depth=N`
   la profondità massima di ricorsione. N=0 indica che la ricerca deve essere effettuata esclusivamente nella cartella di partenza, senza scendere in eventuali sottocartelle. N=1 include anche le sottocartelle dirette della cartella di partenza, e così via. Se l'utente non lo specifica, di default la ricerca è ricorsiva senza alcun limite di profondità.

Lo script dovrà essere eseguito, quindi, con la seguente sintassi:

```
perl findMatch.pl --prefix=PREFIX [--path=PATH] [--depth=N]
```

Una volta eseguito, lo script dovrà cercare, a partire dalla cartella di riferimento (di default o specificata tramite `--path`) e rispettando l'eventuale limite di profondità imposto da --depth, tutti i file e le cartelle il cui nome (non l'intero path, solo il nome finale) inizia con la stringa fornita tramite --prefix. Sia i file che le cartelle devono essere considerati candidati validi.

Lo script stamperà su `STDOUT` il numero di match trovati e il *path relativo alla cartella corrente* di ciascun elemento trovato, uno per riga, in ordine alfabetico.

ESEMPIO:

1. Contenuto della cartella `ProvaTest` (che potrebbe essere situata per esempio in `/home/esame/ProvaTest`):

```
ProvaTest/
|── report.txt
|── repository/
│      |── report2.txt
│      └── deep/
│            └── report3.txt
└── altro.txt
```

2. Esecuzione script (eseguito dalla cartella `/home/esame`):

```
perl findMatch.pl --prefix=rep --depth=1 --path=ProvaTest
```

3. Azioni effettuate internamente dallo script

Lo script cercherà, a partire da `/home/esame/ProvaTest`, scendendo al massimo di 1 livello (quindi includendo `ProvaTest/` stessa e le sue sottocartelle dirette, ma non `ProvaTest/repository/deep/`), tutti i file e le cartelle il cui nome inizia per "`rep`".

4. Output (STDOUT):

```
Trovate 3 corrispondenze per il prefisso rep.
ProvaTest/report.txt
ProvaTest/repository
ProvaTest/repository/report2.txt
```